Higher-Order Functions

DM580: Functional Programming

Casper Bach

SDU

Course Plan and Reading

Week 39	Week 40	Week 41	Week 42
Introduction and Haskell Preliminaries *Read:** Foreword + 1 + 2	Types and Basic Functions *Read:* 3 + 4	Lists and Recursion *Read:* 5 + 6 ***Submit handin assn. 1***	*Fall Break*
Week 43	**Week 44**	**Week 45**	**Week 46**
Higher-Order Functions and Type Classes *Read:* 7 + 8 ***Submit handin assn. 2***	Programming with I/O *Read:* 10 ***Submit handin assn. 3***	Monads *Read:* 12	*No lecture* *Read:* TBA ***Submit handin assn. 4***
Week 47	**Week 48**	**Week 49**	**Week 50**
Concurrency *Read:* Beautiful Concurrency	Lazy Evaluation *Read:* 15 ***Submit handin assn. 5***	Exam Project *Read:* TBA	Q&A Session

* Except where otherwise indicated, reading refers to Graham Hutton's *Programming in Haskell*, second edition

Reading Material

This lecture relates to these chapters of *Programming in Haskell, 2nd ed.*

- Chapter 7

Learning Objectives

- Define higher-order functions and identify them by their type signatures
- Implement generic functions over lists (map, filter, foldr)
- Demonstrate universality of generic recursion scheme (foldr) by using it to define standard functions
- State and apply key laws, such as map fusion

What is a Higher-Order Function?



Higher-Order Functions

Recap, from Week 40

Higher-Order Functions

Recap, from Week 40

- A higher-order function is a function that treats other functions as values



Higher-Order Functions

Recap, from Week 40

- A higher-order function is a function that treats other functions as values
- Example: `map`

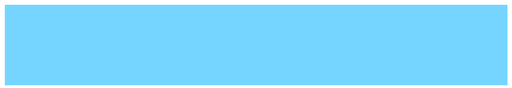


Higher-Order Functions

Recap, from Week 40

- A higher-order function is a function that treats other functions as values
- Example: `map`

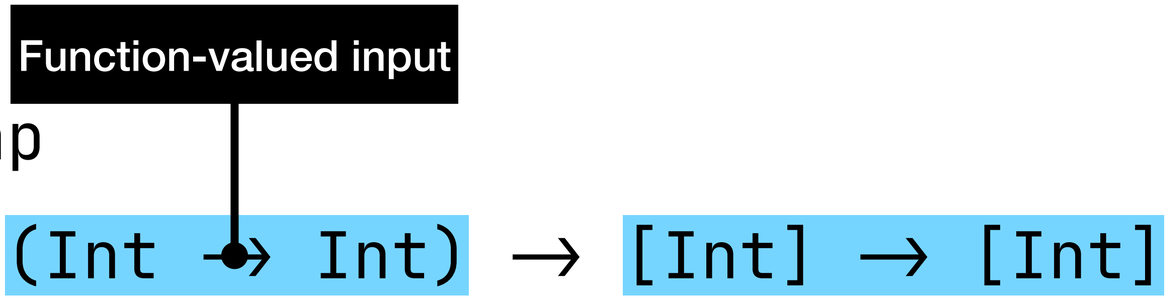
```
map :: (Int → Int) → [Int] → [Int]
```



Higher-Order Functions

Recap, from Week 40

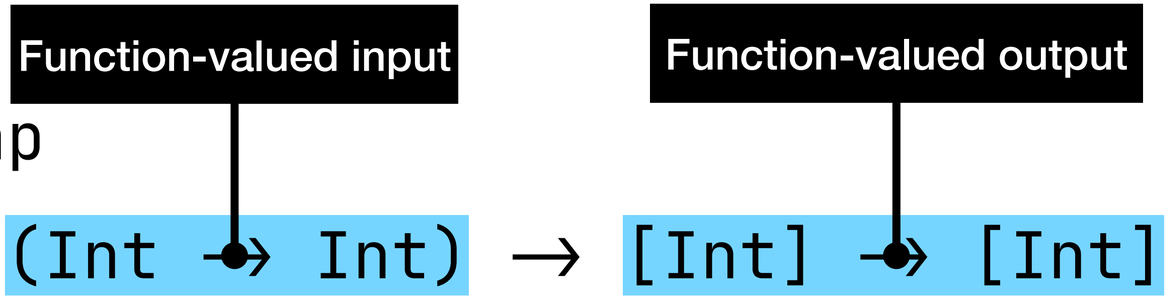
- A higher-order function is a function that treats other functions as values
- Example: `map`



Higher-Order Functions

Recap, from Week 40

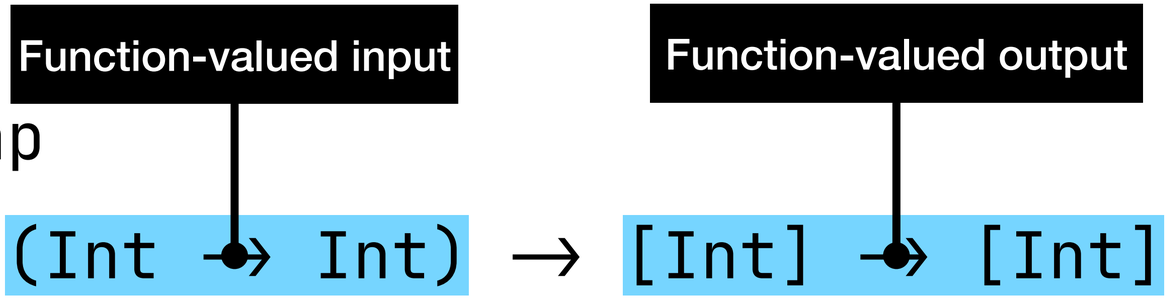
- A higher-order function is a function that treats other functions as values
- Example: `map`



Higher-Order Functions

Recap, from Week 40

- A higher-order function is a function that treats other functions as values
- Example: `map`

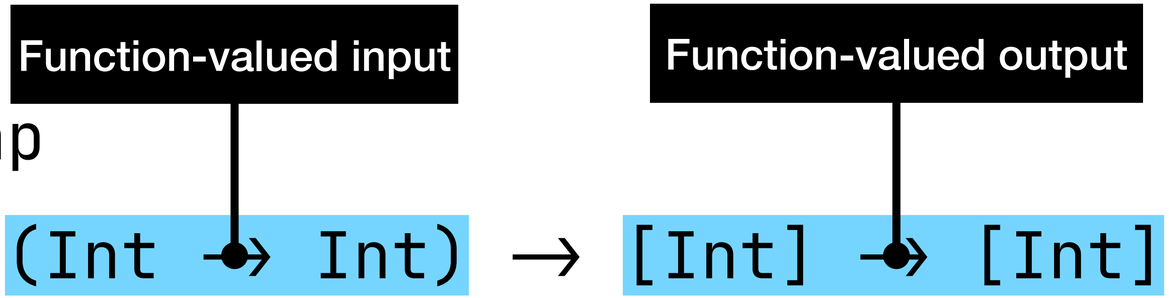


- Example: λ (written \ in Haskell)

Higher-Order Functions

Recap, from Week 40

- A higher-order function is a function that treats other functions as values
- Example: `map`



- Example: λ (written \ in Haskell)

```
incremented :: [Int] → [Int]
incremented xs = map (\x → x + 1) xs
```



Why Higher-Order Functions?

Why Higher-Order Functions?

Why Higher-Order Functions?

- Programmers had an itch: the same loop ten times, one line different

Why Higher-Order Functions?

- Programmers had an itch: the same loop ten times, one line different
  - ```
    sortByTitle(books);
    sortByAuthor(books); // same loop, one line different
    sortByYear(books);   // same loop, one line different
    ```
    ```

Why Higher-Order Functions?

- Programmers had an itch: the same loop ten times, one line different
  - ```
    sortByTitle(books);
    sortByAuthor(books); // same loop, one line different
    sortByYear(books);   // same loop, one line different
    ```
- The cure for the itch: *higher-order functions.*
    ```

Why Higher-Order Functions?

- Programmers had an itch: the same loop ten times, one line different
  - ```
    sortByTitle(books);
    sortByAuthor(books); // same loop, one line different
    sortByYear(books);   // same loop, one line different
    ```
- The cure for the itch: *higher-order functions.*
  - Make the difference an argument: a function
    ```

Why Higher-Order Functions?

- Programmers had an itch: the same loop ten times, one line different
  - ```
    sortByTitle(books);
    sortByAuthor(books); // same loop, one line different
    sortByYear(books);   // same loop, one line different
    ```
- The cure for the itch: *higher-order functions.*
  - Make the difference an argument: a function
  - `sortBy :: (Book → Book → Ordering) → [Book] → [Book]`
    ```

Why Higher-Order Functions?

- Programmers had an itch: the same loop ten times, one line different
  - ```
    sortByTitle(books);
    sortByAuthor(books); // same loop, one line different
    sortByYear(books);   // same loop, one line different
    ```
- The cure for the itch: *higher-order functions.*
  - Make the difference an argument: a function
  - ```
    sortBy :: (Book → Book → Ordering) → [Book] → [Book]
    ```
  - ```
    sortBy byTitle  books
    sortBy byAuthor books
    sortBy byYear   books
    ```
    ```

Why Higher-Order Functions, Continued

Why Higher-Order Functions, Continued

- Programmers still had an itch
  - `sum    :: [Int] → Int`
  - `length :: [a] → Int`
  - `and    :: [Bool] → Bool`

Why Higher-Order Functions, Continued

- Programmers still had an itch
  - `sum    :: [Int] → Int`
  - `length :: [a] → Int`
  - `and    :: [Bool] → Bool`
- All follow the same recursive structure. Repetitive to implement:

Schema

```
f []     = v
f (x:xs) = x # f xs
```

Why Higher-Order Functions, Continued

- Programmers still had an itch
  - `sum    :: [Int] → Int`
  - `length :: [a] → Int`
  - `and    :: [Bool] → Bool`
- All follow the same recursive structure. Repetitive to implement:

```
sum []     = 0
sum (x:xs) = x + sum xs
```

Schema

```
f []     = v
f (x:xs) = x # f xs
```

Why Higher-Order Functions, Continued

- Programmers still had an itch
  - `sum    :: [Int] → Int`
  - `length :: [a] → Int`
  - `and    :: [Bool] → Bool`
- All follow the same recursive structure. Repetitive to implement:

Schema

```
f []     = v
f (x:xs) = x # f xs
```

```
sum []     = 0
sum (x:xs) = x + sum xs
```

```
length []     = 0
length (x:xs) = 1 + length xs
```

Why Higher-Order Functions, Continued

- Programmers still had an itch
  - `sum :: [Int] → Int`
  - `length :: [a] → Int`
  - `and :: [Bool] → Bool`
- All follow the same recursive structure. Repetitive to implement:

Schema

```
f []     = v
f (x:xs) = x # f xs
```

```
sum []     = 0
sum (x:xs) = x + sum xs
```

```
length []     = 0
length (x:xs) = 1 + length xs
```

```
and []     = True
and (x:xs) = x && and xs
```

Why Higher-Order Functions, Continued

- Programmers still had an itch
  - `sum    :: [Int] → Int`
  - `length :: [a] → Int`
  - `and    :: [Bool] → Bool`
- All follow the same recursive structure. Repetitive to implement:

Schema

```
f []     = v
f (x:xs) = x # f xs
```

```
sum []     = 0
sum (x:xs) = x + sum xs
```

```
length []     = 0
length (x:xs) = 1 + length xs
```

```
and []     = True
and (x:xs) = x && and xs
```

- Could we optimize the following? How/why?

Why Higher-Order Functions, Continued

- Programmers still had an itch
  - `sum    :: [Int] → Int`
  - `length :: [a] → Int`
  - `and    :: [Bool] → Bool`
- All follow the same recursive structure. Repetitive to implement:

Schema

```
f []     = v
f (x:xs) = x # f xs
```

```
sum []     = 0
sum (x:xs) = x + sum xs
```

```
length []     = 0
length (x:xs) = 1 + length xs
```

```
and []     = True
and (x:xs) = x && and xs
```

- Could we optimize the following? How/why?

```
sum . map (*2)
```

Why Higher-Order Functions, Continued

- Programmers still had an itch
  - `sum    :: [Int] → Int`
  - `length :: [a] → Int`
  - `and    :: [Bool] → Bool`
- All follow the same recursive structure. Repetitive to implement:

Schema

```
f []     = v
f (x:xs) = x # f xs
```

```
sum []     = 0
sum (x:xs) = x + sum xs
```

```
length []     = 0
length (x:xs) = 1 + length xs
```

```
and []     = True
and (x:xs) = x && and xs
```

- Could we optimize the following? How/why?

```
sum . map (*2)
```

```
length . map (+2)
```

Why Higher-Order Functions, Continued

- Programmers still had an itch
  - `sum    :: [Int] → Int`
  - `length :: [a] → Int`
  - `and    :: [Bool] → Bool`
- All follow the same recursive structure. Repetitive to implement:

Schema

```
f []     = v
f (x:xs) = x # f xs
```

```
sum []     = 0
sum (x:xs) = x + sum xs
```

```
length []     = 0
length (x:xs) = 1 + length xs
```

```
and []     = True
and (x:xs) = x && and xs
```

- Could we optimize the following? How/why?

```
sum . map (*2)
```

```
length . map (+2)
```

```
and . map not
```

Why Higher-Order Functions, Continued

- Programmers still had an itch
  - `sum    :: [Int] → Int`
  - `length :: [a] → Int`
  - `and    :: [Bool] → Bool`
- All follow the same recursive structure, using the same *recursion scheme* (`foldr`).

```
sum = foldr (+) 0
```

```
length = foldr (\_ → (+1)) 0
```

```
and = foldr (&&) True
```

- Optimization, by fusion:

Schema

```
foldr f z . map g
==  foldr (\x r → f (g x) r) z
```

Why Higher-Order Functions, Continued

- Programmers still had an itch
  - `sum    :: [Int] → Int`
  - `length :: [a] → Int`
  - `and    :: [Bool] → Bool`
- All follow the same recursive structure, using the same *recursion scheme* (`foldr`).

```
sum = foldr (+) 0
```

```
length = foldr (\_ → (+1)) 0
```

```
and = foldr (&&) True
```

Schema

```
foldr f z . map g
==  foldr (\x r → f (g x) r) z
```

- Optimization, by fusion:

```
foldr (+) 0 . map (*2)
== foldr (\x r → 2*x + r) 0
```

Why Higher-Order Functions, Continued

- Programmers still had an itch
  - `sum    :: [Int] → Int`
  - `length :: [a] → Int`
  - `and    :: [Bool] → Bool`
- All follow the same recursive structure, using the same *recursion scheme* (`foldr`).

```
sum = foldr (+) 0
```

```
length = foldr (\_ → (+1)) 0
```

```
and = foldr (&&) True
```

Schema

```
foldr f z . map g
==  foldr (\x r → f (g x) r) z
```

- Optimization, by fusion:

```
foldr (+) 0 . map (*2)
== foldr (\x r → 2*x + r) 0
```

```
foldr (\_ → (+1)) 0 . map (+2)
== foldr (\_ → (+1)) 0
```

Why Higher-Order Functions, Continued

- Programmers still had an itch
  - `sum    :: [Int] → Int`
  - `length :: [a] → Int`
  - `and    :: [Bool] → Bool`
- All follow the same recursive structure, using the same *recursion scheme* (`foldr`).

```
sum = foldr (+) 0
```

```
length = foldr (\_ → (+1)) 0
```

```
and = foldr (&&) True
```

Schema

```
foldr f z . map g
==  foldr (\x r → f (g x) r) z
```

- Optimization, by fusion:

```
foldr (+) 0 . map (*2)
== foldr (\x r → 2*x + r) 0
```

```
foldr (\_ → (+1)) 0 . map (+2)
== foldr (\_ → (+1)) 0
```

```
foldr (&&) True . map not
== foldr (\x r → not x && r) True
```

Higher-Order Functions — Abstraction and Reuse

Higher-Order Functions — Abstraction and Reuse

- Higher-order functions let us abstract over common patterns

Higher-Order Functions — Abstraction and Reuse

- Higher-order functions let us abstract over common patterns
- Such as factoring a common comparison
  - `sortByTitle books` ⇝ `sortBy byTitle books`

Higher-Order Functions — Abstraction and Reuse

- Higher-order functions let us abstract over common patterns
- Such as factoring a common comparison
  - `sortByTitle books` ⇝ `sortBy byTitle books`
- Or even factoring a common recursion
  - `sum [] = 0; sum (x:xs) = x + sum xs` ⇝ `foldr (+) 0`

Higher-Order Functions — Abstraction and Reuse

- Higher-order functions let us abstract over common patterns
- Such as factoring a common comparison
  - `sortByTitle books` ⇝ `sortBy byTitle books`
- Or even factoring a common recursion
  - `sum [] = 0; sum (x:xs) = x + sum xs` ⇝ `foldr (+) 0`
- Such factoring affords reuse of code; but also reuse of laws to prove optimizations correct. E.g., the three optimizations, that follow from the same law
  - `foldr f z . map g` = `foldr (\x r → f (g x) r) z`



Implementing Higher-Order Functions

Programming Exercise 1: Write Map

```
map :: (a → b) → [a] → [b]
map f []     = _
map f (x:xs) = _
```

Programming Exercise 2: Filter

```
filter :: (a → Bool) → [a] → [a]
filter p []     = _
filter p (x:xs) = _
```

Programming Exercise 3: Foldr

```
filter :: (a → Bool) → [a] → [a]
filter p = foldr _ _
```



Generic Recursion

Programming Exercise: Filter as a Foldr

```
foldr :: (a → b → b) → b → [a] → b
foldr g z []     = _
foldr g z (x:xs) = _
```

Laws



Selected Laws

Can be proven and derived on a case-by-case basis. Many arise from *initial algebra semantics*

- `map` laws
  - `map id x = x`
  - `map f (map g xs) = map (f . g) xs`
- some `foldr` laws
  - `foldr f z (xs ++ ys) = foldr f (foldr f z ys) xs`
  - `foldr (:) [] xs = xs`
- `filter` fusion
  - `filter p . filter q = filter (\x → q x && p x)`
- A general fusion property can be stated about foldr which subsumes many of the laws and properties considered here. Unfortunately beyond the scope of this course.

Fusion in Haskell

- Not implemented in the compiler directly
- Instead specified as rewrite rules in libraries
  - `{-# RULES "map/map" forall f g xs. map f (map g xs) = map (f . g) xs #-}`
- See, e.g.,
  - Richard Bird & Oege de Moor. *Algebra of Programming.* Prentice Hall, 1997.
  - Graham Hutton. *A tutorial on the universality and expressiveness of fold.* JFP, 1999.
  - Duncan Coutts, Ralf Hinze, Andres Löh. *Stream fusion: from lists to streams to nothing at all.* ICFP 2007.
  - Thomas Harper. *A Library Writer's Guide to Shortcut Fusion.* Haskell 2011.